

DALLAS ELASTIC USER GROUP • MAY 19, 2026



Building an AI-Powered Medical Research Assistant with Elasticsearch

Hybrid retrieval + multi-agent orchestration for clinical research

Mohan Krishna Alavala

Senior DevOps Engineer, Albertsons Companies

github.com/mohankrishnaalavala

What is MedSearch AI?

A medical research assistant — and why I'm sharing it tonight

What it does

- ▶ Answers medical questions in plain English
- ▶ Searches across PubMed, ClinicalTrials.gov, and FDA drug data
- ▶ Gives back a written answer with citations you can verify
- ▶ Open-source — runs end-to-end on one small VM

Why this talk

- ▶ Real, shipped hackathon project — not a slide-only demo
- ▶ Lessons on retrieval that broke, and how I fixed them
- ▶ Patterns transfer to any domain corpus you're working with
- ▶ Plenty of time for Q&A — usually the most useful part

The Problem

Medical research is drowning in data — and racing the clock

1.5M+

new medical articles published
every year

15-20 hrs

to do a traditional literature
review

4+ sources

PubMed, ClinicalTrials.gov, FDA,
journals

Keyword-only

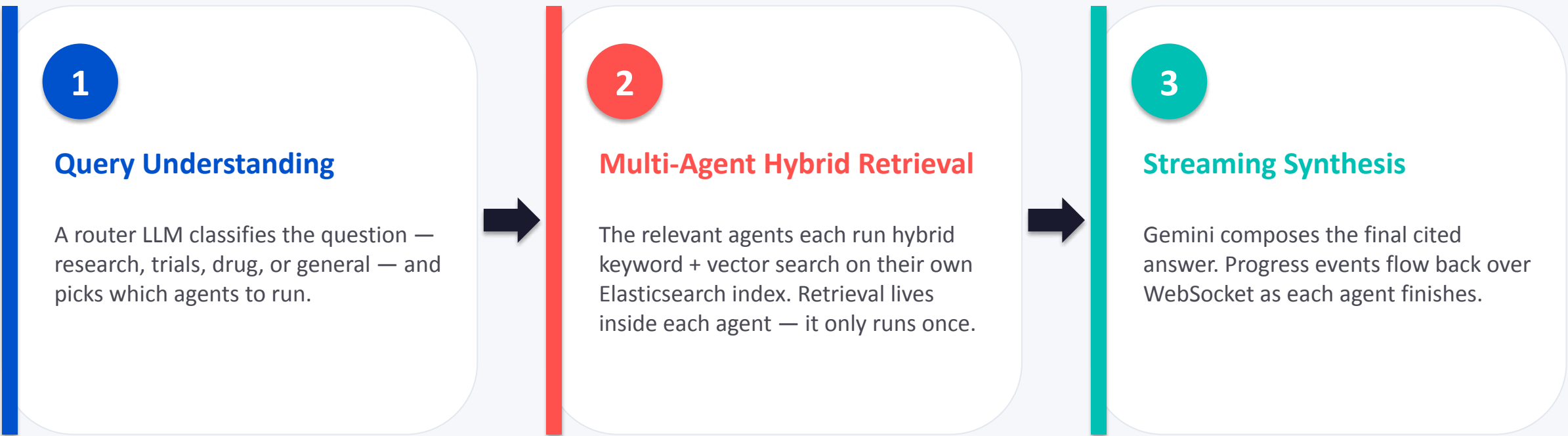
search misses semantically
related findings

Result

Delayed clinical decisions, missed connections across studies, and hours of high-skill time spent on retrieval instead of reasoning.

MedSearch AI — In One Slide

How a question becomes a cited answer — three steps



Why Elasticsearch?

Hybrid retrieval, filters, and observability — in one engine

Hybrid in one engine

BM25 keyword search + dense vectors + structured filters, all in one query DSL — no second store to babysit.

Per-source indices

Separate indices for PubMed, trials, and drugs — each with its own mappings and scoring.

Production-grade filters

Date ranges, trial phase, approval status — composable with kNN.

Observability included

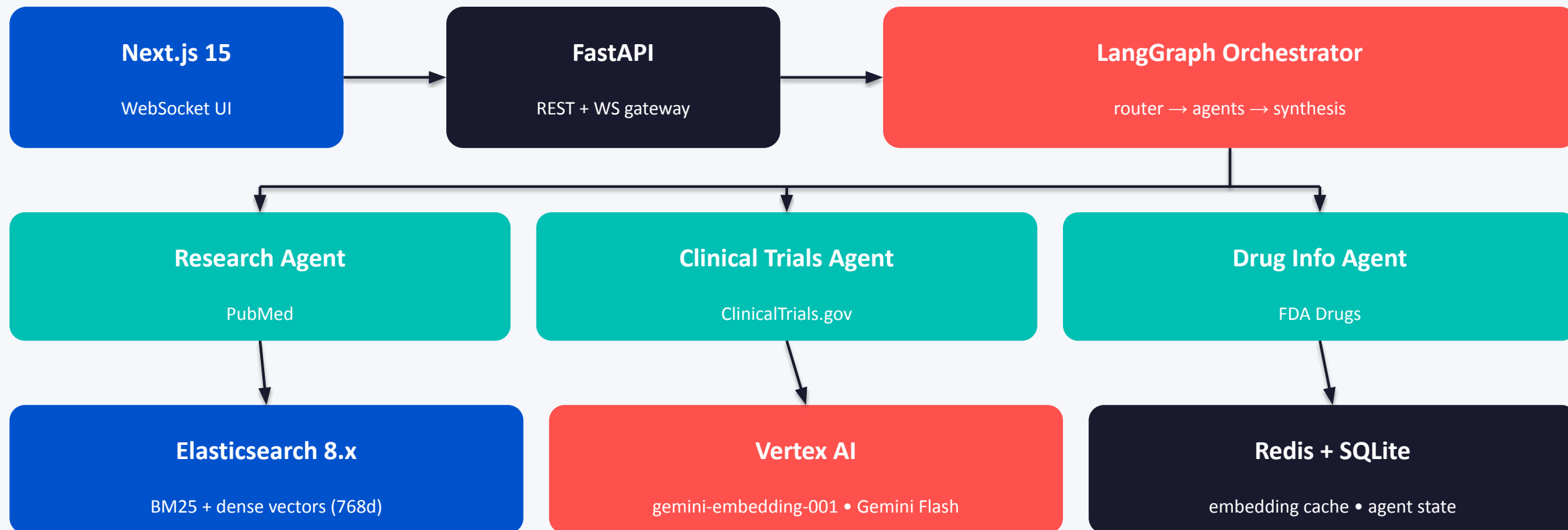
Elastic APM and Kibana — same stack handles retrieval, LLM calls, and dashboards.

How hybrid scoring works

Default 70% semantic + 30% keyword (configurable per environment). Filters compose with kNN. Optional Gemini reranker on top-K when the result set is borderline.

Architecture

Next.js → FastAPI + LangGraph → Elasticsearch + Vertex AI



Hosted on a single GCE e2-standard-2 VM, Docker Compose, Nginx + Let's Encrypt for HTTPS/WSS.

Performance & Resilience

Sub-3-second answers — even when something downstream is sad

Caching strategy

- Redis embedding cache → fewer Vertex AI calls
- Search result cache with TTL for repeat queries
- allkeys-LRU eviction, 512MB ceiling
- Falls through cleanly if Redis is down

Graceful degradation

- Agents fall back to curated mock data
- Users still get cited answers if ES is unavailable
- Auto-reconnect when dependencies recover
- Health checks + restart policies on every container

Observability

- Elastic APM auto-instruments FastAPI + agents
- Transactions, errors, dependency map in Kibana
- 10% sampling default, tunable per env
- Same stack catches retrieval + LLM regressions

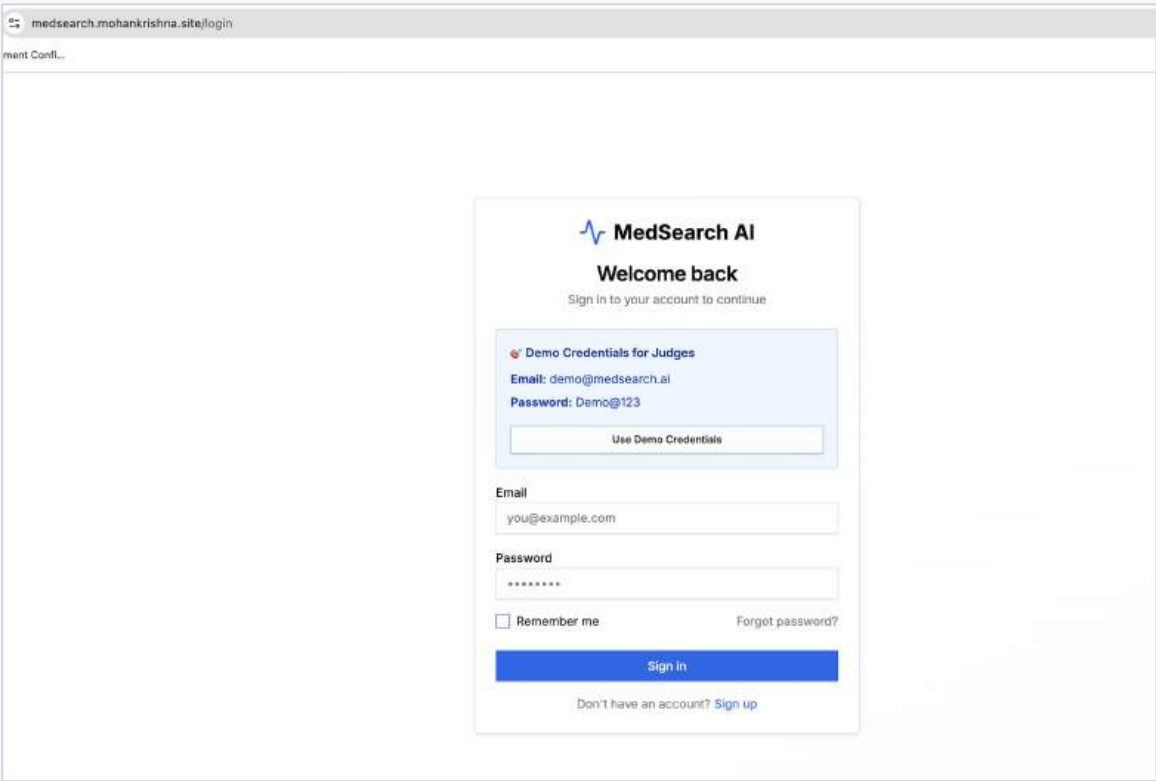
Numbers we hit

- 95%+ citation accuracy with confidence scores
- <3s end-to-end on warm cache
- Single e2-standard-2 VM (8GB / 2 vCPU)
- ES 2.5GB • API 1.5GB • Frontend 512MB

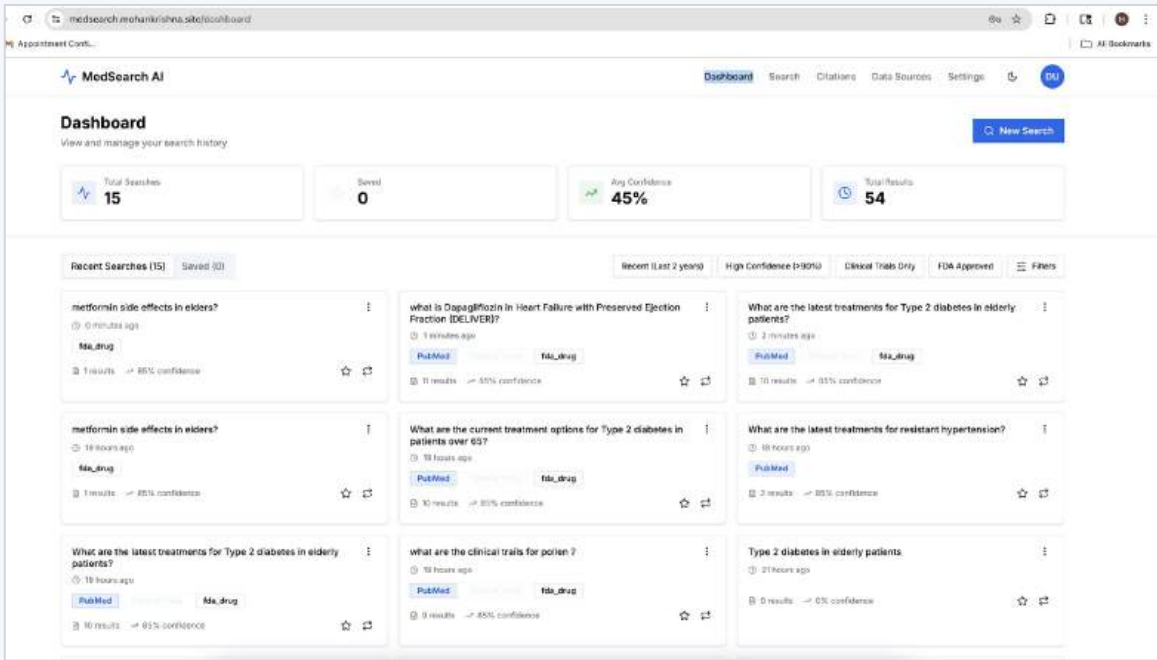
Product Walkthrough

From login to a streaming, cited answer

1. Sign in



2. Dashboard



Built with Next.js 15 + shadcn/ui — WebSocket streaming wired into TanStack Query.

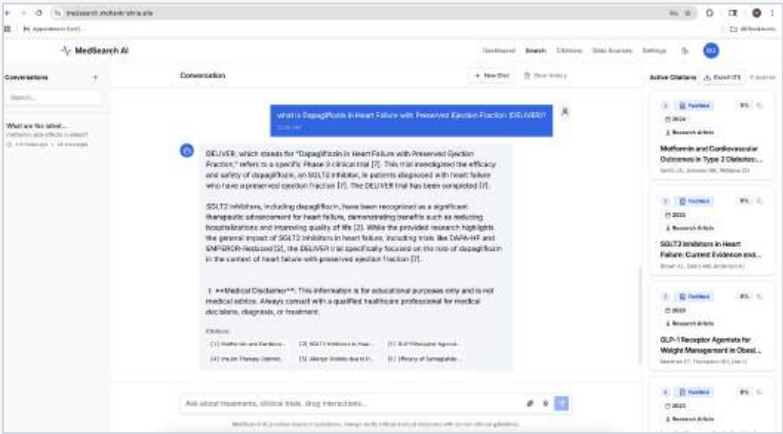
MedSearch in Action

Two real queries — a research question, and a drug + trial deep dive

1. Treatment guidelines query



2. Drug + trial deep dive

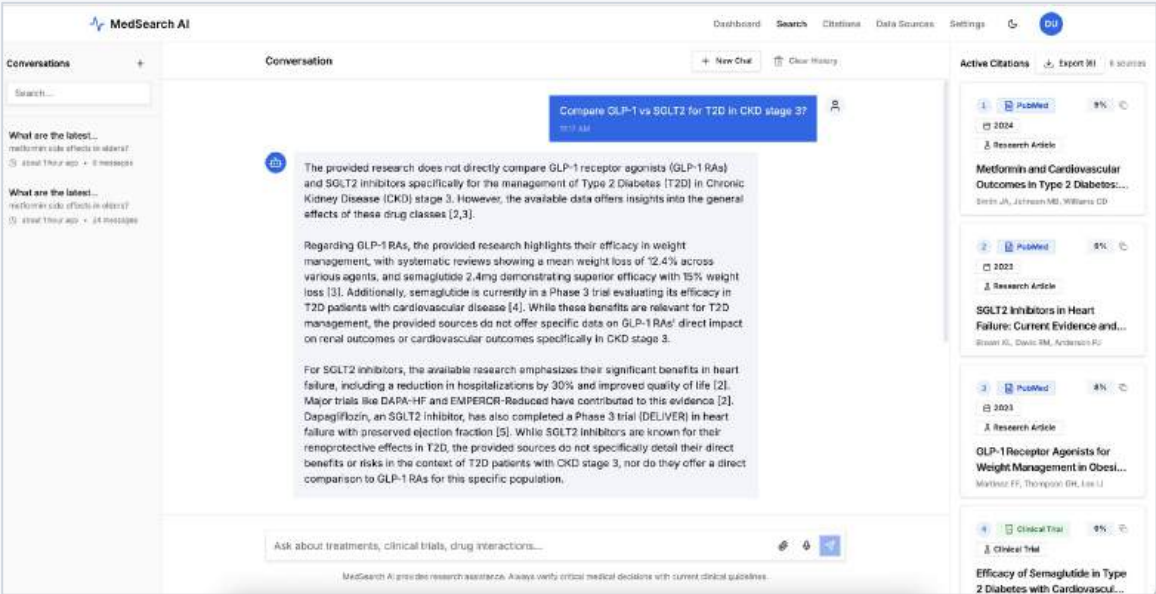


Streaming partials make it feel ~3x faster than the actual end-to-end latency.

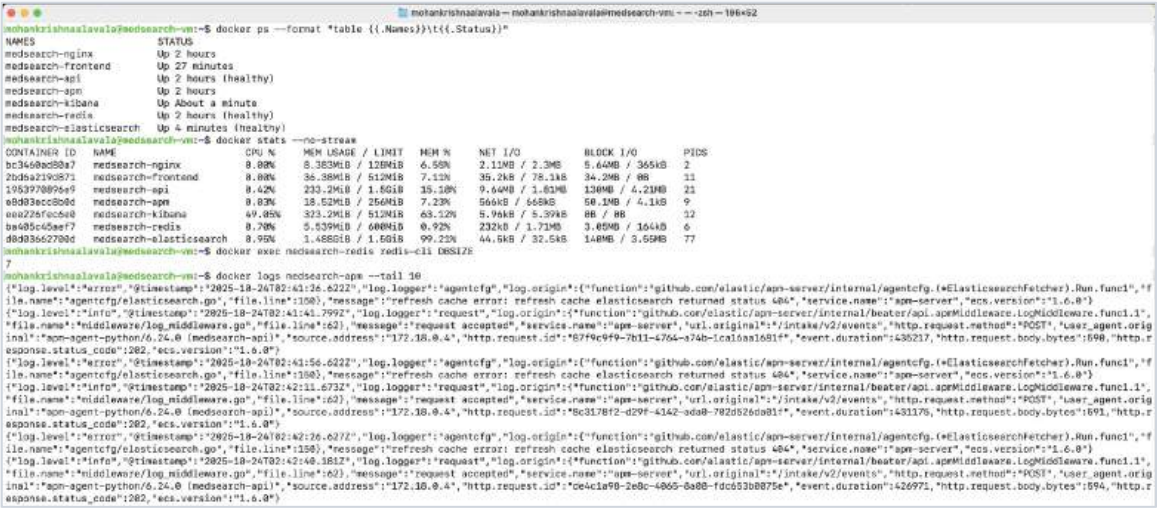
Compare Mode + Observability

Comparing two studies side-by-side, with Elastic APM watching every call

Compare research articles



Elastic APM — server traces



APM caught a vector dimension mismatch in seconds — would otherwise have looked like "the model got worse".

What Worked

Decisions I'd repeat next time

- **Per-source indices over one big index**
Specialized mappings, filters, and scoring per data source — clearer ownership and faster queries.
- **Lean on hybrid from day one**
Skipping pure-vector saved a class of hallucinated near-misses on medical acronyms.
- **Stream early, synthesize last**
Showing partial agent results over WebSocket made the UX feel ~3x faster than the actual latency.
- **Mock data fallback inside agents**
Hackathon judges hit the demo at the same time as a routine ES restart — answers still came back cited.
- **Elastic APM as the safety net**
Caught a vector dimension mismatch in seconds that would otherwise have looked like "the model got worse".

Challenges & Lessons

The parts I won't blog about — but you should hear them anyway

Tuning hybrid weights

The 70/30 split is the **current** sweet spot — not a universal one.
Use a labeled query set; don't tune by vibes.
Re-tune when the corpus shifts.

LLM cost vs. quality

Reranker is great but adds latency and \$.
Made it a flag (VERTEX_AI_RERANK_ENABLED).
Cap top-k; rerank only when confidence is borderline.

Citations are a feature, not a footer

Synthesis must cite **what it actually used**, not what looks plausible.
Prompted with retrieved IDs and validated post-hoc.
Confidence score per claim builds user trust.

Single VM, real users

Right-size memory per container or ES will OOM first.
Nginx + WSS needs careful timeouts for streaming.
Health checks + restart policies aren't optional.

Key Takeaways

If you remember three things...

01

Hybrid > pure vector

BM25 + vectors fused in Elasticsearch beat either approach for medical jargon.

02

Agents need an opinion

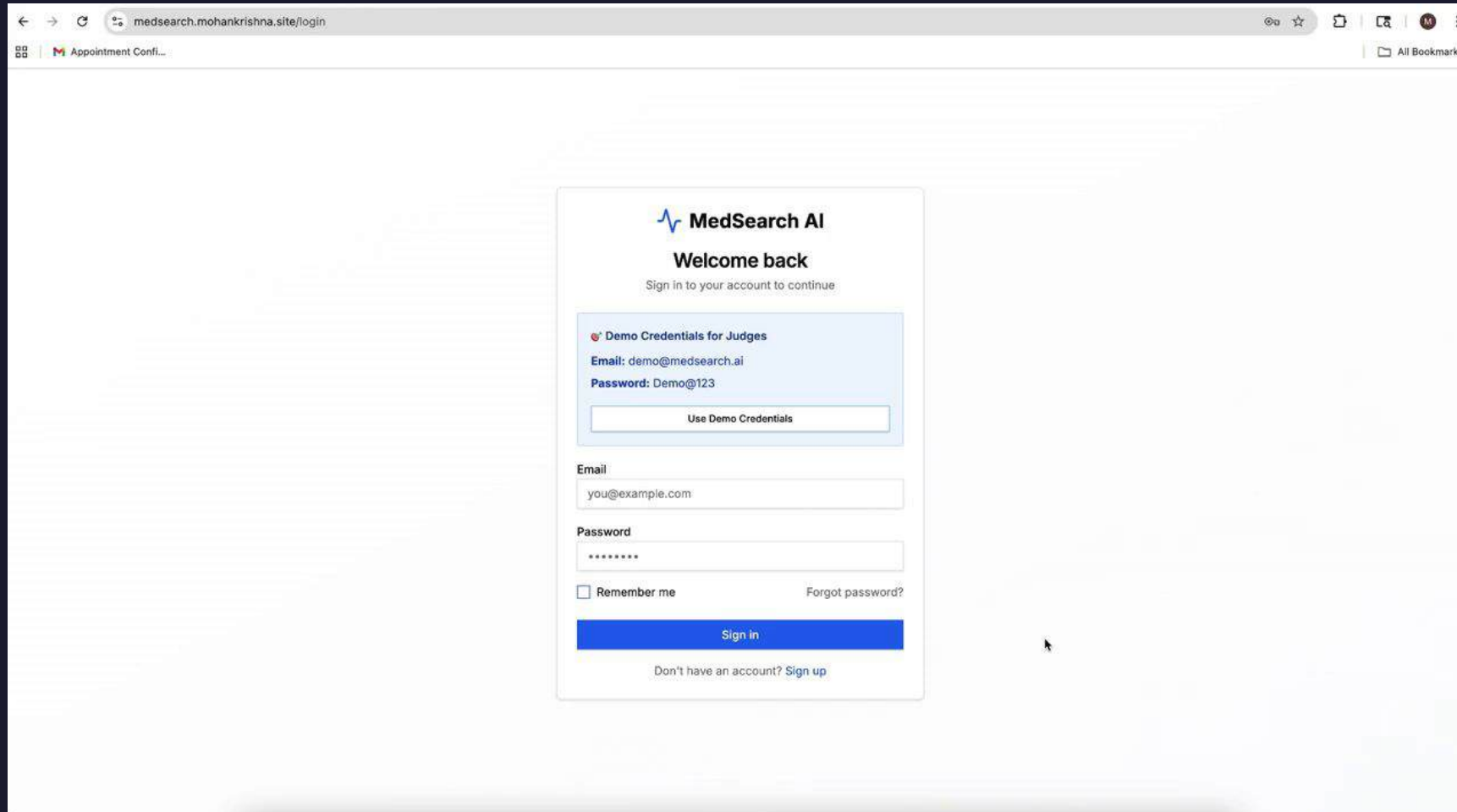
Specialized agents tied to specialized indices outperform a single generalist.

03

Resilience is a feature

Graceful degradation, caching, and APM are what turn a demo into something usable.

A 2× speed replay of the live demo — happy to take questions



The screenshot shows a web browser window with the address bar displaying "medsearch.mohankrishna.site/login". The page features a login form for "MedSearch AI". At the top of the form, it says "Welcome back" and "Sign in to your account to continue". Below this, there is a section for "Demo Credentials for Judges" with the email "demo@medsearch.ai" and password "Demo@123", and a button labeled "Use Demo Credentials". Underneath, there are input fields for "Email" (containing "you@example.com") and "Password" (masked with dots). There is a "Remember me" checkbox and a "Forgot password?" link. A blue "Sign in" button is at the bottom of the form, with a link "Don't have an account? Sign up" below it.

medsearch.mohankrishna.site/login

Appointment Confi...

All Bookmarks

MedSearch AI

Welcome back

Sign in to your account to continue

Demo Credentials for Judges

Email: demo@medsearch.ai

Password: Demo@123

Use Demo Credentials

Email

you@example.com

Password

☐ Remember me [Forgot password?](#)

Sign in

Don't have an account? [Sign up](#)